

ISW - Resumen P2

* **Testing de Software (08):** Actividad del SCM que busca destruir el software, a través de un proceso que involucra encontrar defectos (**cuya presencia se asume, siempre habrá**). Se debe ir con una actitud negativa para demostrar que algo es incorrecto

- **Asegurar vs Control de Calidad:** Una vez definidos los requerimientos, debo tener en cuenta que la calidad no puede inyectarse al final del proyecto, depende de actividades realizadas a lo largo de todo el proceso.
 - La detección de errores tempranamente ahorra tiempo, esfuerzos y costo.
 - La calidad no solo es del producto, sino del proceso
 - Sin embargo, el testing **NO** asegura calidad por sí mismo

° **Error o Defecto Humano:** **Error** es una equivocación humana durante el análisis, diseño, programación o pruebas. **Defecto** es el problema que queda incorporado en el software como consecuencia de ese error humano; pasa a escala de product

- **Severidad:** **Bloqueante, Crítico, Mayor, Menor, Cosmético**
- **Prioridad:** **Urgencia, Alta, Media, Baja**

* **Niveles de Testing:**

- **Testing Unitario:** Se prueba cada componente tras su realización, solo se prueban componentes individuales. Los errores suelen ser reparados apenas se encuentran sin necesidad de documentarlos
- **Testing de Integración:** Se prueban componentes en conjunto interactuando. **DEBE SER INCREMENTAL**, (top-down / bottom-up) Los módulos críticos son los probados más tempranamente posible. Minimizar programas auxiliares y Conectar de a poco partes complejas
- **Testing de Sistema:** Busca determinar si el sistema en su totalidad funciona como lo esperado (Fallas, Seguridad, Protección, Stress, Performance). Se deben investigar tanto **RF** como **RNF**
- **Testing de Aplicación:** Prueba realizada por el usuario para ver si la app se ajusta a sus necesidades. Encontrar defectos **NO ES EL FOCO**, sino establecer confianza en el sistema, las partes del sistema o las características específicas o no funcionales del sistema. Comprende las pruebas de laboratorio como de entorno real

* **Casos de Prueba:** Tienen como objetivo descubrir errores en forma completa con el menor esfuerzo y tiempo posible. Son un conjunto de condiciones o variables bajo las cuales un tester determinará si un programa está funcionando o no. Una

buena definición de casos de prueba nos ayuda a REPRODUCIR defectos. **Unidad de Actividad de Prueba**

- Derivación de Casos de Prueba: Desde Documentos de Cliente -> Información de Relevamiento -> Requerimientos -> Especificaciones de Programación -> Desde el Código
- **Respecto a los Casos**: - Ninguna técnica es completa - Las técnicas atacan distintos problemas - Combinar las técnicas es mejor - Sin requerimientos todo es más difícil - Es mejor ser sistemático y documentar las suposiciones
- **Condiciones de Prueba**: Esta es la reacción esperada de un sistema frente a un estímulo particular, está constituido por diferentes entradas. Una condición de prueba debe ser probada por al menos un caso de prueba

Elementos de un Caso de Prueba:

- Objetivo: La característica del sistema a comprobar
- Datos de entrada y ambiente: Datos a introducir al sistema que se encuentra en condiciones preestablecidas
- Comportamiento Esperado: La salida o acción esperada en el sistema de acuerdo a los requerimientos del mismo

Estrategias de Casos de Prueba:

- Caja Negra: **Basado en especificaciones**, Partición de equivalencias, análisis de valores límites, etc. **Basados en Experiencias**, adivinanzas de defectos, testing exploratorio
- Caja Blanca: Se basan en el análisis de la estructura interna del software o un componente del software (Código). Se puede garantizar el testing coverage. Cobertura de caminos básicos, sentencias, decisión, condición, múltiple condición

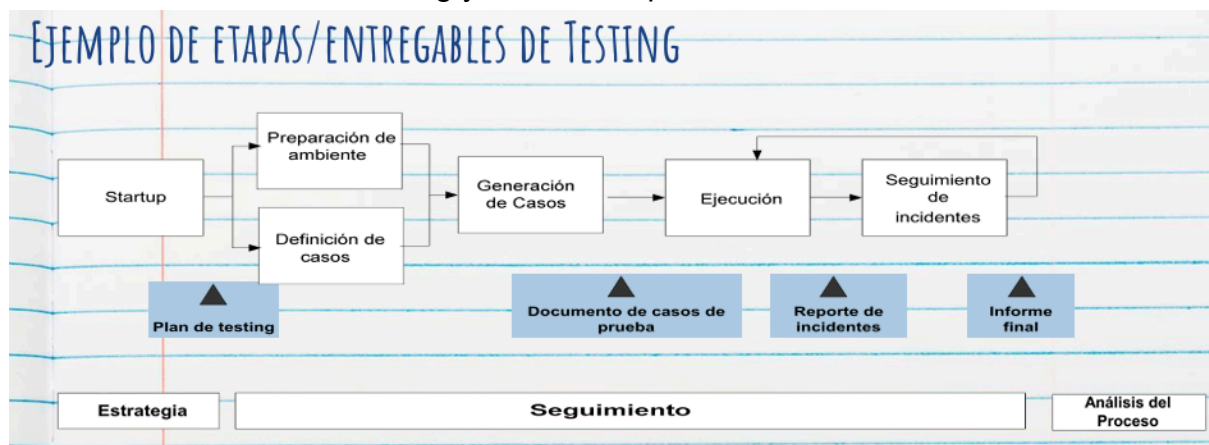
Ciclo de Test: Abarca la totalidad de la ejecución de casos de prueba establecidos aplicados a una misma versión del sistema a probar.

Regresión: Al concluir un ciclo de pruebas y reemplazarse la versión del sistema sometido al mismo, debe realizarse una verificación total de la nueva versión; a fines de prevenir la introducción de nuevos defectos al solucionar los detectados

* **Proceso del Testing** Planificación -> **Identificación** -> **Especificación** -> Ejecución (Si hay fallas regresa) -> **Análisis de Fallas** (Regresa si hay que seguir probando) -> Fin de las Pruebas

- **Planificación**: Actividad de verificar que se entienden las metas y objetivos del cliente, las partes interesadas (stakeholders), el proyecto, y los riesgos de las pruebas que se pretende abordar
- Construcción del Test Plan: Riesgos, objetivos y estrategias de testing, recursos, criterio de aceptación

- **Controlar:** Revisar los resultados del testing, test coverage, C.A. y toma de decisiones
- **Identificación y Especificación:** - Revisión de la base de pruebas - Verificación de las para el software bajo pruebas - Evaluar testabilidad de requerimientos de sistema - Identificar datos necesarios - Diseño y priorización de casos de prueba - Diseño del entorno de prueba
- **Ejecución:** - Desarrollar y dar prioridad a nuestros casos de prueba - Crear datos de prueba - Automatizar lo necesario - Creación de conjuntos de C.D.P - Registrar resultado de pruebas como versiones de herramientas de SW - Comparar resultados con los esperados
- **Evaluación y Reporte:** - Evaluar los C.A. - Reporte de pruebas de user para stakeholders - Recolección de info para actividades de prueba - Verificación de entregables y defectos corregidos - Evaluación de resultado de actividades de testing y lecciones aprendidas



Importancia del Testing: No se necesita tener código para iniciar a testear. Ni es para probar que el SW funciona, no es igual a calidad de producto ni proceso. Es importante para:

- Evitar ser demandado por el cliente
- Para reducir riesgos
- Porque los defectos en SW son inevitables
- Para construir la confianza en mi producto
- Las fallas son costosas
- Para verificar que el SW se ajusta a los requerimientos y validar que las funciones se implementan correctamente

* **Testing de Ciclo de vida de SW:**

- Dar visibilidad de manera temprana al equipo, de cómo se va a probar el producto
- Disminuir los costos de correcciones de defectos

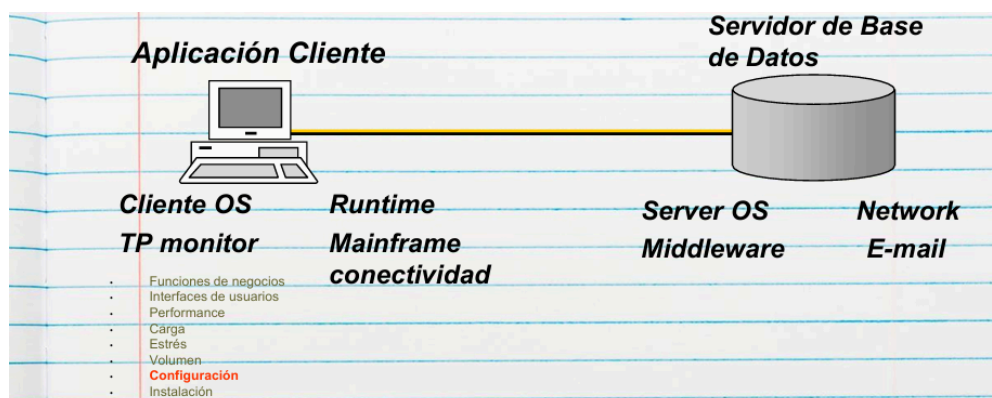
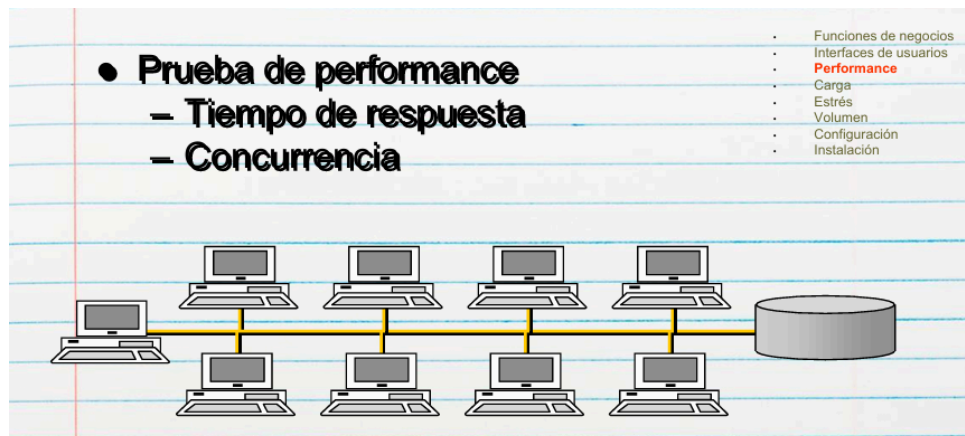
El testing es suficiente cuando se tiene la confianza de que el sistema funciona correctamente, y depende del riesgo del sistema. El testing exhaustivo es imposible. Los criterios de aceptación son los usualmente determinantes de indicar si un testing ha finalizado. Se puede definir en términos de: - Costos - % de test sin falla -

Fallas predichas que siguen en el SW - No hay defectos de determinada severidad en el SW

- **Principios del Testing:** El testing muestra presencia de defectos, es mejor si se hace temprano y es independiente del contexto. No existe la ausencia de defectos
 - Un programador **NO** debería testear su código
 - Una unidad de programación **NO** debería testear sus desarrollos
 - Examinar el software para: Ver que haga lo que debe, y NO haga lo que no
 - No planificar el esfuerzo de testear sobre que no se van a encontrar defectos
- **Psicología del Testing:**
 - Smoke Test: Primer run de un SW que prueba que no se genera una falla catastrófica

Tipos de Pruebas

- **Testing Funcional:** Se basan en pruebas y características y su interoperabilidad con sistemas específicos. **Basado en Requerimientos y Pruebas de Negocio**
- **Testing No Funcional:** Carga, Stress, Usabilidad, Mantenimiento, Fiabilidad, Portabilidad
- Pruebas de GUI / Pruebas de Configuración / Pruebas de Performance



* **Testing de Software AGILE(09):** Se propone que las pruebas no sean una etapa final del proyecto, sino una prueba continua integrada en el desarrollo.
Buscando: - Detectar defectos temprano - Feedback Veloz - Mejorar Calidad

Tradicional Vs Agile: En **Tradicional**, el testing ocurre al final, las etapas están separadas, hay mucha documentación y el feedback es tardío. Los testers trabajan aparte de los desarrolladores. En **Agile** El testing ocurre en todo el desarrollo, integrando developers y testers, iteraciones cortas, feedback constante y alta automatización

***Principios de Agile:**

- Testing se mueve hacia adelante en el proyecto
- Testing no es una fase
- TODOS hacen testing
- Reducir la latencia del feedback
- Las pruebas representan expectativas
- Mantener el código limpio, corregir los defectos rápido
- Reducir la sobrecarga de documentación de las pruebas
- Las pruebas son parte del “done”
- De probar al final a Conducido por Pruebas

***Prácticas Concretas:**

- **Pruebas unitarias automatizadas:** Verifican pequeñas partes del código automáticamente.
- **TDD (Test Driven Development):** Primero se escribe la prueba y luego el código.
- **Pruebas de regresión automatizadas:** Permiten verificar que cambios nuevos no rompan funcionalidades existentes.
- **ATDD (Acceptance Test Driven Development):** Las pruebas de aceptación se definen antes del desarrollo, basadas en requerimientos del negocio.
- **Pruebas exploratorias:** El tester explora el sistema libremente buscando comportamientos inesperados.
- **Versionado de pruebas junto al código:** Las pruebas se gestionan en el mismo control de versiones que el software.

Rol del tester en Agile: Deja de solo encontrar defectos y pasa a:

- Colaborar con developers - Participar en planificación - Ayudar a definir criterios de aceptación - Automatizar pruebas - Mejorar Procesos

Cambios Culturales: - Más comunicación - Trabajo colaborativo - Adaptación al cambio - Responsabilidad compartida sobre la calidad

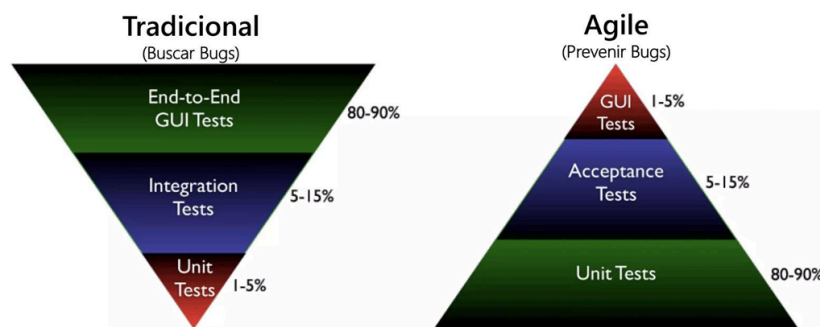
Cuadrantes del Testing:



Pirámide del Testing:

En Agile se prioriza automatizar las pruebas más rápidas y frecuentes

Beneficios del Testing Automatizado:



- Ejecución Rápida - Reutilización - Más cobertura, precisión y alcance en pruebas - Reducción de Costos

* Filosofía LEAN y KANBAN(10):

Lean: La filosofía Lean es un enfoque de gestión que busca maximizar el valor entregado al cliente mediante la mejora continua y la optimización de los procesos.

- Su núcleo fundamental consiste en identificar y eliminar sistemáticamente todo tipo de desperdicio, entendiendo como tal a cualquier actividad, inventario o sobreprocesamiento que consuma recursos sin aportar valor directo al producto o servicio final

*Principios:

- **Eliminar Desperdicios:** Identificar y remover todo aquello que no aporta valor
- **Amplificar Aprendizaje:** Potenciar el conocimiento continuo dentro del proceso.
- **Diferir compromisos:** Retrasar las decisiones irreversibles hasta el último momento posible para contar con mayor información.

- **Entregar lo antes posible**: Optimizar los tiempos para agilizar las entregas al cliente.
- **Dar poder al equipo**: Brindar autonomía y confianza a las personas que ejecutan el trabajo.
- **Embeber la Integridad conceptual**: Asegurar la coherencia y calidad interna del producto o servicio.
- **Ver el todo**: Comprender el sistema de manera global y no solo de forma aislada

Tipos de Desperdicio: 2 Grandes tipos

- **Necesario**: Aquel que no añade valor directo pero se debe **minimizar** porque es requerido por el proceso actual
- **Puro Desperdicio**: Aquel que no añade valor en absoluto y se debe **evitar** por completo

7 + 1 Desperdicios: Tradicional vs Conocimiento

Desperdicios en Manufactura	Desperdicios en servicios basados en conocimiento
Inventario	Trabajo parcialmente terminado
Sobreproducción	Características extra
Proceso extra	Proceso extra
Transporte	Cambio de tareas Transferencia de conocimiento
Esperas	Esperas/Demoras
Movimiento	Cambio de contextos (Seteo)
Defectos	Defectos
Talento no utilizado	Talento no utilizado

KANBAN: Es un método para definir, gestionar y mejorar servicios que entregan trabajo del conocimiento, tales como servicios profesionales, trabajos o actividades en las que interviene la creatividad y el diseño tanto de productos de software como físicos.

Valores de Kanban: - Transparencia - Equilibrio - Colaboración - Foco en Cliente - Flujo - Liderazgo - Entendimiento - Acuerdo - Respeto

***Principios:**

- **Gestión del Cambio Evolutivo**: Identificar lo que se realiza actualmente. Entender los procesos como tal respetando roles, responsabilidades y puestos vigentes.
 - Acordar la búsqueda de mejora a través de cambio evolutivo
 - Fomentar Actos de Liderazgo a todos los niveles
- **Entrega de Servicios**: Comprender y enfocarse en cumplir las expectativas y necesidades del cliente.
 - Dejar que los trabajadores se auto organicen en torno al trabajo

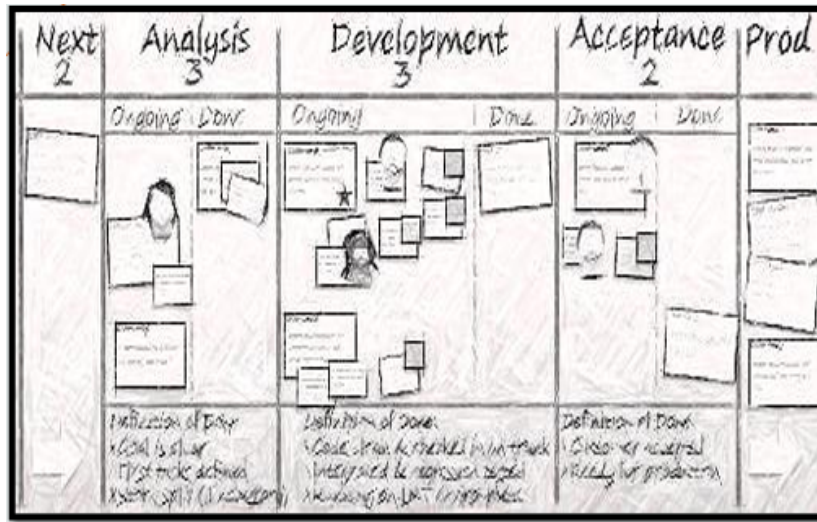
- Revisar periódicamente la red de servicios para mejorar los resultados entregados

***Prácticas Generales:**

- **Visualizar**: Construir un modelo visual (tablero) que muestre el trabajo, su flujo y sus riesgos. Esto permite absorber mucha información rápido, unifica la visión del equipo y agiliza las decisiones colaborativas
- **Limitar el Trabajo en Curso (WIP - Work In Progress)**: Asignar límites explícitos a la cantidad de ítems permitidos en progreso por cada estado. Se basa en la premisa "¡Deja de empezar, empieza a terminar!". Al no saturar el sistema al 100% de su capacidad, se genera la "holgura" necesaria para que el trabajo fluya de manera suave y predictiva (evitando el efecto de una autopista en hora pico). Limitar el WIP provoca conversación y mejora ante los bloqueos
- **Gestionar el Flujo**: Monitorear el movimiento del trabajo de principio a fin centrándose en el cliente. La medición del flujo aporta datos valiosos para predecir entregas, ser confiables y gestionar expectativas
 - El flujo debe ser pull, no push; el trabajo avanza de derecha a izquierda basándose en la capacidad disponible
- **Hacer las Políticas Explícitas**: Definir acuerdos claros, visibles y consensuados por todas las partes (clientes, interesados y equipo). Deben ser pocas, sencillas, aplicables en todo momento y fácilmente modificables por quienes prestan el servicio
- **Establecer Ciclos de Retroalimentación**: Implementar reuniones con la cadencia adecuada para fomentar el aprendizaje, la colaboración y la mejora continua basándose en datos
- **Mejorar Colaborativamente**: Impulsar cambios mediante el método científico, diseñando experimentos basados en hipótesis dentro de entornos seguros (donde fallar sea seguro/safe-to-fail). Si funciona se mantiene, y si no, se vuelve fácilmente al estado anterior

¿Cómo aplicar Kanban?

- + **Proceso**: modelar nuestro proceso.
- + **Trabajo**: decidir la unidad de trabajo.
- + **Límites de WIP**: limitar el WIP para ayudar al flujo de trabajo.
- + **Política**: definir políticas de calidad.
- + **Cuellos de Botella y Flujo**: mover recursos a los cuellos de botella.
- + **Clase de Servicio**: diferentes trabajos tienen diferentes políticas – definición de hecho ("done"), para cada estado.
- + **Cadencia**: Releases, planificaciones, revisiones



Cola de Producto	Análisis		Desarrollo		Listo para Build	En Testing		En Producción
	En progreso	Hecho	En progreso	Hecho		En Progreso	Listo para Despliegue	
Casos de Uso 60 %								
Mantenimiento 30 %								
Defectos 10 %								

Cadencias:

- **Team Kanban Meeting (Diaria):** Su propósito es observar y seguir el estado y el flujo del trabajo (no de las personas). Busca responder cómo entregar los elementos más rápido, ver si hay capacidad disponible y definir qué tomar a continuación
- **Team Replenishment Meeting (Semanal o a demanda):** Sirve para seleccionar los próximos elementos de la lista de trabajo (backlog) que van a ingresar al tablero
- **Team Retrospective (Quincenal o mensual):** Espacio para reflexionar sobre cómo el equipo gestiona su trabajo y acordar cómo pueden mejorar

* **TDD (Test Driven Development):** Es una metodología y técnica avanzada de desarrollo cuyo objetivo principal es escribir primero las pruebas unitarias y luego el código de producción

- Surge de 2 grandes prácticas:
 - Test-first development: Escribir las pruebas primero
 - Refactoring: Refactorización constante del código

Beneficios:

- **Código robusto, seguro y limpio**: El software es más predecible y es fácil saber cuándo está realmente terminado
- **Tolerancia al cambio**: Al tener una red de seguridad (las pruebas), el código se vuelve más fácil de modificar y más barato de mantener
- **Mejor arquitectura**: Promueve componentes con alta cohesión y bajo acoplamiento, obligando al desarrollador a pensar primero en el diseño de las APIs
- **Velocidad a largo plazo**: Asegura la cobertura de pruebas, reduce la duplicación de código y, con la práctica, acelera la velocidad de desarrollo facilitando el despliegue continuo

***El ciclo Red-Green-Refactor**: Ciclo iterativo que consta de tres pasos fundamentales:

- **RED (Falla)**: Se escribe una prueba unitaria para una funcionalidad mínima que aún no existe. Al ejecutarla, la prueba debe fallar (mostrarse en rojo) o ni siquiera compilar
- **GREEN (Pasa)**: Se escribe la cantidad mínima de código de producción necesaria para que la prueba pase (se muestre en verde). No importa si el código no es perfecto, lo crucial es que funcione
- **REFACTOR (Refactorizar)**: Se limpia y optimiza el código recién escrito, eliminando duplicaciones o complejidades innecesarias, asegurando que la prueba siga pasando (manteniéndose en verde)

***3 Leyes Principales:**

- No se permite escribir código de producción sin antes tener un caso de prueba que falle
- No se debe escribir más de una prueba de la necesaria para que ocurra un fallo
- No se debe escribir más código de producción que el estrictamente necesario para superar la prueba que falla

Tradicional Vs TDD:

- **Desarrollo Tradicional**: Se diseña, se desarrolla el código y al final se escriben las pruebas. Esto genera un control de calidad **reactivo**
- **TDD**: Se diseñan las pruebas, se ejecutan (fallan), se desarrolla el código y luego se refactoriza. Esto hace que el control de calidad sea **proactivo**

Patrones de TDD

Patrones *red bar*

- One Step Test: Escribe un solo test que verifique una pieza mínima de funcionalidad.
- Starter Test: Primer test que te ayuda a explorar la API o comportamiento deseado.
- Explanation test: Escribe comentarios o preguntas para aclarar qué esperas que haga el código
- Learning test: permite conocer APIs externas sobre las que no sabemos su comportamiento.
- Another Test: Añade un nuevo test para cubrir un caso adicional o una variante del comportamiento.
- Regression Test: Agrega un test que reproduce un bug descubierto, para evitar regresiones futuras.

Patrones *green bar*

- Fake it: Devuelve un valor fijo o solución "falsa" solo para hacer pasar el test.
- Triangulación: Introduces un **segundo test con diferentes datos** para forzar una implementación más general
- Obvious implementation: Cuando la solución es trivial y obvia, puedes implementarla directamente sin "Fake It".
- One to many: Patrón para resolver test para colecciones de elementos.

Refactorizar: Transformar el código interno de manera disciplinada sin alterar su comportamiento externo (todas las pruebas deben seguir pasando)

1. Asegurarse de que las pruebas actuales pasen.
2. Identificar código que "huela mal" (mal estructurado o complejo).
3. Determinar cómo simplificarlo y realizar los cambios.
4. Volver a ejecutar las pruebas para verificar que todo siga funcionando igual.

* **Métricas en los diferentes Enfoques(11):**

***Tradicional:** Métricas de:

Proceso -> Proyecto -> Producto

- Las métricas de **Proyecto** se consolidan para crear métricas de **Proceso**

Métricas de Proyecto: - Tamaño de Producto - Esfuerzo - Tiempo Calendario -

Defectos

Tipos de Mediciones:

- **Desarrollador:**
 - **Esfuerzo**, duración estimada y actual
 - Porcentaje de cobertura del test
 - Número y tipo de defectos encontrados en test y revisión por pares
- **Organización:**
 - **Tiempo Calendario**
 - Performance actual y planificada de esfuerzo y presupuesto
 - Precisión de estimaciones y esfuerzo
 - **Defectos en release**
- **Equipo de Desarrollo:**

- **Tamaño de Producto**
- Duración estimada y real entre hitos
- Niveles de staff actual y estimado
- Distribución de esfuerzo
- Estatus y Volatilidad de Requerimientos
- Defectos encontrados en integración y prueba de sistemas
- Defectos encontrados en peer reviews
- Porcentaje Tests ejecutados

***Agile: La Medición es una Salida NO ACTIVIDAD. Medir lo necesario y Nada más.**

- La mayor prioridad es satisfacer al cliente por entregas tempranas y continuas de software valioso en funcionamiento
- El SW funcional, es la medida de progreso

Métricas Ágiles:

1. **Velocidad:** Mide los story points completados en cada iteración
2. **Capacidad:** Mide y planifica la disponibilidad del equipo para cada Sprint. Se calcula cruzando las horas disponibles totales del equipo con los puntos de historia que se comprometen a realizar
3. **RTF (Running Tested Features):** Mide la cantidad de características o funcionalidades que ya están desarrolladas, probadas y funcionando iteración por iteración

***Kanban: Métricas Clave:**

- **Cycle Time:** Se mide el tiempo desde el inicio de un proceso hasta terminarlo. Se suele medir en días/esfuerzo. Mide la capacidad y ritmo de terminación
- **Lead Time:** Mide desde que el cliente pide el item de trabajo hasta la entrega. Mide el tiempo de entrega global
- **Touch Time:** Mide el tiempo neto real donde se estuvo trabajando en el item, quitando tiempo de buffer, cola, etc

$$\text{Touch Time} \leq \text{Cycle Time} \leq \text{Lead Time}$$

- **Eficiencia del Ciclo de Progreso:** Se divide el Touch Time sobre el tiempo total transcurrido

$$\% \text{ Eficiencia ciclo proceso} = \frac{\text{Touch Time}}{\text{Elapsed Time}}$$

***Aseguramiento de Calidad de Proceso y de Producto (PPQA)(12):**

***Calidad:** Aspectos y características de un producto o servicio que se relacionan con su habilidad de alcanzar las necesidades manifiestas. Suele ser relativa a circunstancias o el cliente

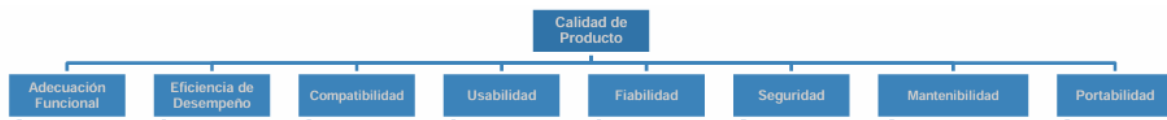
Un SW de Calidad Satisface:

Expectativas de Cliente/Usuario -> Necesidad de Gerencia -> Necesidad de Equipo de Desarrollo

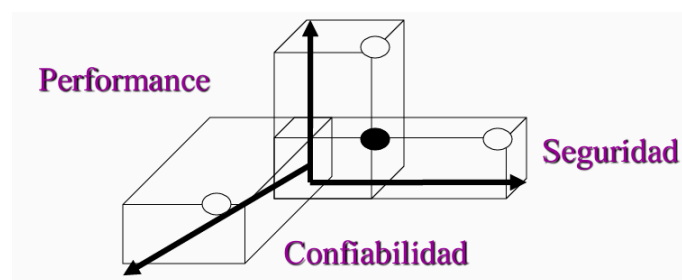
La Calidad no se inyecta, sino se construye y embebe; Las personas son la clave para lograrlo, debe estar planificada. **Calidad Necesaria/Programada/Realizada -> Esperada**

***Calidad de Producto:** Modelo de calidad de Uso/Producto/Datos

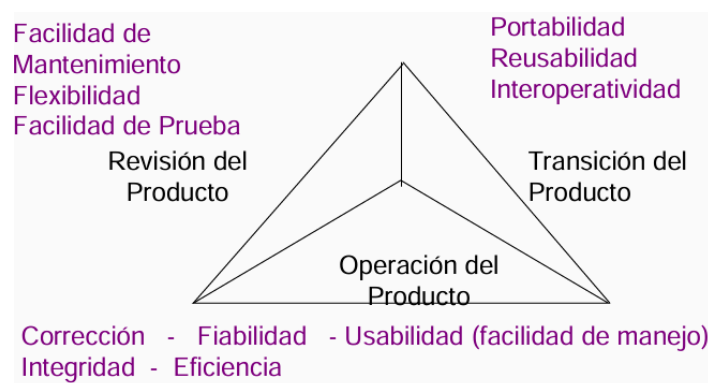
Modelo ISO 25010:



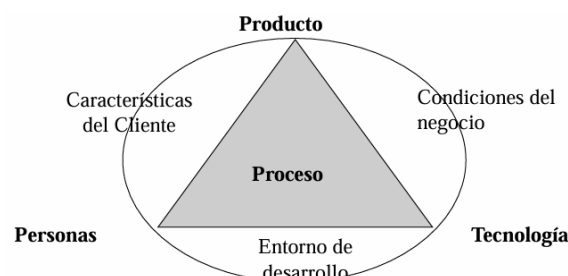
Modelo de Barbacci: Propone Equilibrar Performance/Seguridad y Confiabilidad



Modelo de McCall:



***Calidad y Proceso de Desarrollo:** El proceso es el único factor "controlable" al mejorar la calidad del software y su rendimiento como organización



***Administración de la Calidad del SW:** Definición de estándares y procesos de calidad apropiados y asegurar que los mismos sean respetados

- Debería desarrollarse una cultura de responsabilidad de buscar la calidad por todos

Actividades de la Admin. de Cal. SW:

- **Aseguramiento de Calidad:** Establecer estándares y procedimientos. Asegurar el cumplimiento de los estándares y procedimientos para el software y el proceso. Asegurar que los defectos en el producto, proceso o estándares son informados a la gerencia para que puedan ser solucionados
- **Planificación de Calidad:** Selecciona los procedimientos y estándares aplicables para un proyecto en particular y los modifica si fuera necesario. Un plan de calidad define los productos de calidad deseados y como serán evaluados , y define los atributos de calidad más significativos
- **Control de Calidad:** Asegura que los procedimientos y estándares son respetados por el equipo de desarrollo de software. Existen dos enfoques para el control de calidad: Revisiones de Calidad; Evaluaciones de Software Automáticas y mediciones

Funciones de la Admin. de Cal. SW:

- Prácticas de Aseguramiento de calidad: Desarrollo de herramientas adecuadas, técnicas, métodos y estándares
- Evaluación de: **Planificación de Proyecto de SW - Requerimientos - Proceso de Diseño - Prácticas de Programación - Proceso de Integración y Prueba de SW - Control - Procedimientos**

***Estándares:**

Los procesos estándares deberían ser:

- Cómo deberían conducirse revisiones
- Cómo debería realizarse la administración de configuración

Estos son la clave para la efectiva admin. de calidad, pueden ser:

- Estándares de Producto: Características de componentes
- Estándares de Proceso: Definen como deberían ser implementados los procesos de SW

Modelos de Mejora de Proceso:

- **SPICE:** Software Process Improvement Capability Evaluation
- **IDEAL:** Initiating, Diagnosing, Establishing, Acting, Leveraging

***Modelo de Calidad CMMI (Capability Maturity Model Integration)**

Uno de los modelos más implementados en todo el mundo. No es una norma, y no se “certifica”, sólo se evalúa a través de profesionales reconocidos por el SEI como Lead Appraisers

Constelaciones:

- CMMI-DEV: Provee la guía para medir, monitorear y administrar los procesos de desarrollo

- CMMI-SVC: Provee guía para entregar servicios internos o externos
- CMMI-ACQ: Guía para administrar y adquirir productos y servicios

Representación:

- **Por Etapas:** Niveles de 1 a 5 que indican madurez organizacional
 - **1: Inicial:** Los procesos son impredecibles, reactivos y poco controlados. El éxito depende del esfuerzo heroico de individuos particulares y no de una estructura organizacional
 - **2: Administrado:** Los proyectos de la organización aseguran que los procesos se planifiquen, ejecuten, midan y controlen. Se gestionan activamente los requerimientos, los costos, los cronogramas y la calidad a nivel de proyecto
 - **3: Definido:** Los procesos están estandarizados, documentados y aprobados a nivel de toda la organización
 - **4: Cuantitativamente Administrado:** Se utilizan métricas estadísticas y cuantitativas para controlar los procesos y predecir el rendimiento
 - **5: Optimizado:** La organización se enfoca en la mejora continua de sus procesos basándose en una comprensión cuantitativa de las causas de variación

Nivel	Categoría			
	Administración de Proyectos	Soporte	Administración de Procesos	Ingeniería
5 Optimizado		• Análisis y Causal y Resolución (CAR)	• Administración de Performance Organizacional (OID)	
4 Cuantitativamente Administrado	• Administración Cuantitativa del Proyecto (QPM)		• Performance del Proceso Organizacional (OPP)	
3 Definido	• Administración de Riesgos (RSKM) • Administración Integrada de Proyectos (IPM)	• Análisis y Resolución de Decisión (DAR)	• Definición del Proceso Organizacional (OPD) • Foco en el Proceso Organizacional (OPF) • Capacitación Organizacional (OT)	• Desarrollo de Requerimientos (RD) • Solución Técnica (TD) • Integración de Producto (PI) • Verificación (VER) • Validación (VAL)
2 Administrado	• Administración de Requerimientos (REQM) • Planificación de Proyectos (PP) • Monitoreo y Control de Proyectos (PMC) • Administración de Acuerdo con el Proveedor (SAM)	• Aseguramiento de calidad de Proceso y de Producto (PPQA) • Administración de Configuración (CM) • Medición y Análisis (MA)		
1 Inicial	Procesos sin definir o improvisados			

Ideas Clave:

- El software puede analizarse desde varias perspectivas: como proceso, como producto, la calidad también
- La calidad es difícil de medir
- El SW como proceso es fundamento para mejorar calidad
- Trabajar con calidad es mas barato que sin ella
- La mejora de procesos en el software ha demostrado retornos de inversión sustanciales

CMMI Agile:

- “Nivel 1”
- Identificar el alcance del trabajo
 - Realizar el trabajo

- “Nivel 2”
- Política Organizacional para planear y ejecutar
 - Requerimientos, objetivos o planes
 - Recursos adecuados
 - Asignar responsabilidad y autoridad
 - Capacitar a las personas

Verde : Da
Negro: Nei
Rojo: De:



“Nivel 3” ▫ Mantener un proceso definido ▫ Medir la performance del proceso	Rojo:
“Nivel 4” ▫ Establecer y mantener objetivos cuantitativos para el proceso ▫ Estabilizar la performance para uno o más subprocesos para determinar su habilidad para alcanzar logros	
“Nivel 5” ▫ Asegurar mejora continua para dar soporte a los objetivos ▫ Identificar y corregir causa raíz de los defectos	

***Revisiones Técnicas(13):** Buscan introducir el concepto de verificación y validación. Es un proceso de ciclo de vida completo. Inicia con las revisiones de los requerimientos y continúa con las revisiones del diseño, inspecciones del código hasta la prueba. Deben implementarse desde inicio, ya que el costo de reparar fallas sube exponencialmente al avanzar el desarrollo

- **Verificación:** Construimos un producto Correcto?
- **Validación:** Construimos un producto Correctamente?
- **Falla:** Error en producto de trabajo
- **Producto de Trabajo:** Salida de actividad de ciclo de vida de desarrollo

Tipos de Fallas:

- **Menor:** Condición no deseable pero no ocasiona falla operacional
 - En Código/Diseño: Violacion a estandares de codificacion que no fallaran pero obstruyen visibilidad
 - En Requerimientos: Un requerimiento improbable
 - En Casos de Prueba: Información clara o esfuerzo de testing innecesario debido a redundancia
- **Mayor:** Puede ocasionar una **falla operacional:**
 - En Codigo: Error Lógico y Estructural
 - En Requerimientos: Expresión ambigua, que no refleja la necesidad del cliente
 - En Diseño: Expresión de diseño mal planteada que podría ocasionar falla operacional
 - En Casos de Prueba: Condición que permita no encontrar fallas, o que un test no pueda repetirse
- **Cosméticas:**
 - Errores de Típo/Ortografía/Gramaticales
 - Actualizar plantilla de documento
 - Actualizar Historia de Revisiones

***Principios de Validacion y Verificacion:**

- Prevenir que Curar / Evitar que Eliminar
- Retroalimentación es efectiva
- Priorizar rentabilidad / No hay perfección

***Revisiones TÉCNICAS:** Detectar Defectos y corregirlos en etapas tempranas de desarrollo. Practicadas en donde la calidad y retrabajo son criticos

- Se aplican en varios momentos del desarrollo
- Los errores suelen pasarse a su creador
- No requiere ejecución

Ventajas: Pueden descubrirse muchos errores / Inspeccionar versiones incompletas / Considerarse atributos de calidad

Desventajas: Difícil de formalizar inspecciones / Sobrecargan al inicio los costos y conducen a un ahorro después de que los equipos adquieran experiencia / Requieren tiempo para organizarse y parecen ralentizar el proceso de desarrollo

Costos:

- **Infraestructura:** - Entrenamiento - Desarrollo Informes/Guías - Herramientas de Soporte
- **Operaciones:** - Tiempo Individual y Grupal - Tiempo de Informes
- **Adicionales:** - Preparar materiales - Arreglar calendario - Tiempo mejora de Calidad

Método	Objetivos Típicos	Atributos Típicos
Walkthroughs	Mínima Sobrecarga Capacitación de Desarrolladores Rápido retorno	Poca o ninguna preparación Proceso Informal No hay mediciones No FTR!
Inspecciones	Detectar y remover todos los defectos eficiente y efectivamente	Proceso Formal Checklists Mediciones Fase de Verificación

Tipo de documento	Revisores sugeridos
Arquitectura o Diseño de alto nivel	Arquitecto, analista de requerimientos, diseñador, lider de proyecto, testers.
Diseño detallado	Diseñador, arquitecto, programadores, testers
Planes de proyecto	Líder de proyecto, stakeholders, representante de ventas o marketing, líder técnico, representante del área de calidad,
Especificacion de requerimientos	Analista de requerimientos, líder de proyecto, arquitecto, diseñador, testers, representante de ventas y/o marketing
Código fuente	Programador, diseñador, testers, analista de requerimientos
Plan de testing	Tester, programador, arquitecto, diseñador, representante del área de calidad, analista de requerimientos

Métricas Sugeridas	Fórmula
Densidad de defectos	Total de defectos encontrados / tamaño actual
Total de defectos encontrados	Defectos.Mayor + Defectos.Menor
Esfuerzo de la inspección	Esfuerzo.Planning + Esfuerzo.Preparación + Esfuerzo.reunion + Esfuerzo.Retrabajo
Esfuerzo por defecto	Esfuerzo.Inspeccion / Total de def encontrados
Porcentaje de reinspecciones	Cantidad Reinspecciones / Cantidad Inspecciones
Defectos Corregidos sobre Total de Defectos.	Esfuerzo.Inspeccion / tamaño actual

***Inspección:** Actividad de garantía calidad de SW

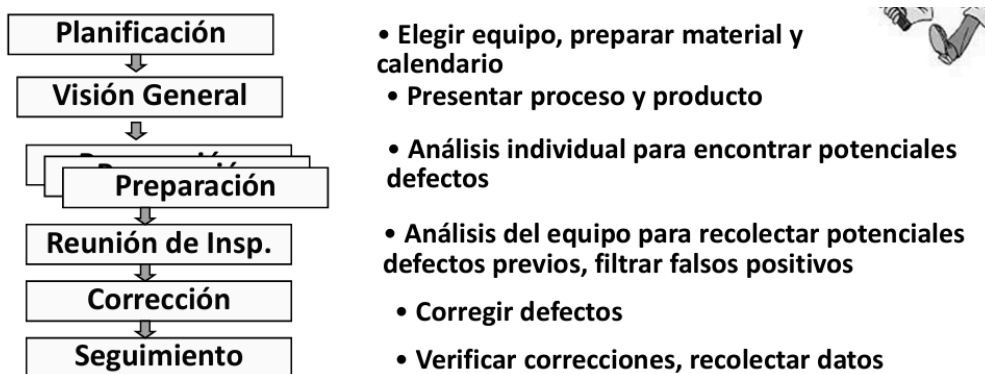
Busca descubrir errores, verificar que el software alcanza sus requisitos, garantizar que el SW cumple ciertos estándares, Hacer proyectos más manejables

SON	NO SON
• La forma más barata y efectiva de encontrar fallas • Una forma de proveer métricas al proyecto • Una buena forma de proveer conocimiento cruzado • Una buena forma de promover el trabajo en grupo • Un método probado para mejorar la calidad del producto	• Utilizadas para encontrar soluciones a las fallas • Usadas para obtener la aprobación de un producto de trabajo • Usadas para evaluar el desempeño de las personas

Roles Principales:

- **Autor:** Encargado de mantener el producto. Asigna un moderador y le entrega el producto. Le reporta el tiempo de retrabajo y nro de defectos a este
- **Moderador:** Planifica y lidera la revisión. Trabaja junto al autor para seleccionar el resto de los roles. Entrega el producto a inspeccionar a inspectores. Hace seguimiento de defectos reportados
- **Lector:** Lee el producto a inspeccionar
- **Anotador:** Registra hallazgos de la revisión
- **Inspector:** Examina el producto antes de la reunión para encontrar defectos y registra tiempos de preparación

Proceso de Inspección:



***Walkthrough:** Técnica de análisis estático en la que un diseñador o programador dirige miembros del equipo de desarrollo y otras partes interesadas a través de un producto de software y los participantes formulan preguntas y realizan comentarios acerca de posibles errores, violación de estándares de desarrollo y otros problemas.

- Buenos resultados, pero se toman pocas métricas
- No hay control del proceso

***Auditoria de Calidad de SW(14):** Evaluación independiente de los productos o procesos de software para asegurar el cumplimiento con estándares, lineamientos, especificaciones y procedimientos, basada en un criterio objetivo incluyendo documentación que especifique:

- Forma o Contenido de productos a desarrollar
- Proceso de Desarrollo
- Cómo medir los estándares o lineamientos

Porque auditar?:

- Brinda una opinión objetiva e independiente
- Permite identificar potenciales áreas de insatisfacción de cliente
- Permite asegurar al cliente que cumplimos sus expectativas
- Permite identificar oportunidades de mejora

Tipos de Auditorías;

- **Auditoría de Proyecto:** alida de manera objetiva la consistencia y evolución del producto a lo largo del proceso de desarrollo. Se realiza según el PACS (Plan de Aseguramiento de Calidad de Software) e incluye inspecciones de software y revisiones de diseño y pruebas.
Verifica las interfaces sean consistentes con la ERS y los requerimientos funcionales se validen en el mismo
- **Auditoría de Configuración Funcional:** Compara el software construido (ejecutables y documentación) con los requerimientos de la ERS. Su propósito es asegurar que el código implemente única y completamente lo descrito en la ERS. El responsable de QA debe validar que la matriz de rastreabilidad esté al día
- **Auditoría de Configuración Física:** Compara el código con la documentación de soporte para asegurar que lo que se va a entregar sea consistente y describa correctamente el desarrollo. El software solo puede entregarse cuando se hayan solucionado las desviaciones encontradas

Roles:

- Gerente de SQA: Prepara el plan de auditorías, calcula sus costos, asigna los recursos y es el responsable final de resolver las no-conformidades
- Auditor: Acuerda la fecha, comunica el alcance, recolecta y analiza evidencia objetiva, ejecuta la auditoría, prepara el reporte y realiza el seguimiento de las acciones acordadas.
- Auditado: Acuerda la fecha, participa activamente, proporciona la evidencia, contesta al reporte de auditoría, propone el plan de acción para las deficiencias y comunica su cumplimiento

***Proceso de Auditoría:** Se divide en 4 grandes fases secuenciales

- 1. Preparación y Planificación:** El auditor solicita la auditoría. Se acuerda lugar, fecha y hora, participantes y alcances. El auditor puede solicitar documentación preliminar
- 2. Ejecución:**
 - Reunion de Apertura: Se presentan los participantes, se revisa el cronograma/alcance, se informa el proceso y carácter formal de la auditoría, y se clarifican dudas o inquietudes.
 - Ejecución: El auditor realiza preguntas y solicita evidencia; el auditado contesta y muestra dicha evidencia. El auditor completa el checklist e informa la fecha de envío del informe
- 3. Análisis y Reporte de Resultado:**
 - Evaluación de Resultados: El auditor revisa el checklist, las anotaciones y la documentación, realiza una verificación cruzada de las desviaciones contra el proceso y redacta el informe preliminar
 - Reunion de Cierre: El auditor presenta y explica el reporte preliminar; se aclaran dudas, se acuerdan los resultados y se define la fecha de entrega del reporte final

- Entrega de Reporte Final: El auditor actualiza el informe con lo hablado en el cierre, lo prepara, guarda una copia y lo envía formalmente al auditado
4. **Seguimiento:** El auditado analiza las desviaciones, diseña un Plan de Acción con fechas de realización y lo envía al auditor. Aprueba el plan de acción. Se ejecutan las acciones e informa su cumplimiento. Verifica el cumplimiento, cierra las desviaciones e informa formalmente el cierre de las mismas

Herramientas y Técnicas:

- **Técnicas:** Checklist, muestreo, revisión de registros y herramientas automatizadas
 - Checklist: Fecha, lista de auditados y sus roles, nombre del auditor, nombre y fase actual del proyecto, objetivo, alcance y la lista de preguntas
- Cuestionario: Iniciar con preguntas abiertas (quién, cuándo, cómo, qué, dónde), hacer preguntas cortas/puntuales y finalizar con preguntas cerradas para clarificar conceptos
- Roles Individuales:
 - Auditor: Escuchar sin interrumpir, observar y manejar adecuadamente el lenguaje corporal, tomar notas, preguntar y repetir lo escuchado para asegurar comprensión
 - Auditado: Ansiedad, tensión, enojo o nerviosismo; sentirse examinado, querer impresionar al auditor, usar el espacio para quejarse de la empresa o brindar demasiada información confidencial de forma involuntaria

Reporte Final:

- **Buenas Prácticas:** Procedimientos desarrollados mucho mejor de lo esperado, cuando el auditado tiene un sistema bastante efectivo y buena cooperación interna
- **Desviación:** Disconformidades del producto respecto a sus requerimientos, falta de control o alteraciones al proceso documentado que generen incertidumbre sobre la calidad. **Requieren obligatoriamente un plan de acción**
- **Observaciones:** Opiniones sobre condiciones o prácticas que incumplen algo menor o que podrían generar una futura desviación. **No requieren un plan de acción**, pero sirven para mejorar

Métricas de Auditoría:

- Esfuerzo por auditoría
- Cantidad de desviaciones
- Duración de auditoría
- Cantidad de desviaciones clasificadas por PA de CMMI

Puntos Clave:

- Las auditorías enfocadas al proceso de desarrollo de software son estrictamente tres: Proyecto, Configuración Física y Configuración Funcional

- Aunque conllevan esfuerzo y costo asociado para los proyectos, sus beneficios a largo plazo son muy superiores
- Constituyen una herramienta fundamental e indispensable para el **Aseguramiento de Calidad en el Software (SQA)**

***Retrospectiva(15):** A intervalos regulares el equipo reflexiona sobre cómo ser más efectivo para a continuación ajustar y perfeccionar su comportamiento en consecuencia. Ocurren al **FINAL** de la iteración, se analizan 3 cosas:

- Lo que salió bien, lo que no hizo y lo que podría mejorarse, a través de una serie de dinámicas

1.Preparar el escenario -> 2.Reunir Datos -> 3.Generar Ideas -> 4.Decidir que Hacer -> 5.Cerrar la Retrospectiva

1. Preparar Escenario: se prepara el equipo para el trabajo que van a hacer en la retrospectiva, puede ser tan simple como la revisión del objetivo, la revisión de la agenda, el registro y la revisión de los acuerdos de trabajo. Cuando el grupo tiene que trabajar más para estar listo para esta fase, utilice alguna de estas actividades
2. Reunir Datos: La recopilación de datos crea una imagen compartida de lo que pasó durante la iteración, release o proyecto. Sin datos, el equipo está especulando sobre qué cambios y mejoras se pueden hacer. Estas actividades ayudan a ver e integrar diferentes tipos de datos
3. Generar Ideas: aquí donde el equipo evalúa los datos y que hacer con la información resultante de ellos. Estas actividades ayudan a que el equipo interprete los datos, los analice, genere ideas y descubra las razones para el cambio
4. Decidir que Hacer: **FUNDAMENTAL**. Mueve el enfoque del equipo para la siguiente iteración. En estas actividades, los miembros del equipo desarrollan propuestas para la acción, identificar las acciones más prioritarias, elaborar planes detallados para experimentos, y establecen metas medibles para alcanzar los resultados. Para iteraciones cortas, no más de 2 o 3 cosas en Dejar de hacer, seguir haciendo y empezar a hacer
5. Cierre: Ofrece momentos para la mejora continua, para reflexionar sobre lo que sucedió durante la retrospectiva, y para expresar agradecimiento. Actividades de otras secciones pueden ser adaptadas para su uso en el cierre

- Retrospectivas y feedback son el corazón de cualquier implementación exitosa Agile / Scrum, el corazón del empirismo
- Elija las actividades de modo tal que los miembros del equipo participen activamente de la sesión
- Las retrospectivas ayudan a los equipos a pensar, decidir y actuar juntos
- Las retrospectivas ayudan a los equipos a mejorar, con el tiempo logran que buenos equipos se conviertan en grandes equipos

No hay posibilidades de mejoras continuas sin este tipo de actividad